

Week 1: Build a Generic Urban Data Integration Platform

1. Data Catalog

To understand the structure, types, and properties of the datasets before building the platform, we documented each into a data catalog.

	Description or Attributes			
Data Catalog Property	Taxi Trips	Weather	Air Quality	Taxi Zone Lookup
Primary Entity	A finished taxi ride	An hourly weather observation	Air quality check for a specific time, location, and pollutant	Taxi zone location metadata
Primary Key	Composite: VendorID + tpep_pickup_date time + PULocationID	Composite: year, month, day, and hour	Composite: Date, Time, Site Num, State Code, County Code	LocationID
Join Attributes	PULocationID, DOLocationID	Hourly timestamp	Date local and time local	LocationID
Temporal Attributes	Tpep_pickup_datetime, tpep_dropoff_datetime	Year, month, day, hour	Date local, Time local, Date GMT, Time GMT, Date of Last Change	None
Attributes Categorical w/ Values	payment_type, RatecodeID, VendorID, PULocationID, DOLocationID, store_and_fwd_flag	Weather condition (coco: 1-14), Cloud cover (cldc: 0-8), Data source indicators	State name, County name, Parameter name, Units of measurement, Method Type	Borough, zone, service zone
Attributes that could Grow	Tpep_pickup_datetime, tpep_dropoff_datetime	Year, month, day, hour	Date local, Time local, Date GMT, Time GMT, Date of Last Change	None

2. Storage Architecture Design

Directory Structure

We implemented a multi-layered Medallion Architecture (Bronze, Silver, Gold) using Delta Lake storage:

- `data/raw/`: Start zone for uncompressed/zipped raw CSV and Parquet files.
- `data/lake/bronze/`: Ingested files stored as Delta tables.
Rejected records are stored alongside them.
- `data/lake/silver/`: Cleaned, normalized, and typed Delta tables.

- `data/lake/gold/`: Enriched analytical tables (e.g., `integrated_taxi_trips`).

Naming Conventions

All schemas follow `snake_case` naming conventions for columns and table names across all layers.

Partitioning Strategy

- Taxi Trips: Partitioned by `taxi_type` and `pickup_date`.
- Weather: Partitioned by `observation_date`.
- Air Quality: Partitioned by `measurement_date`.
- Taxi Zones: Unpartitioned.

Design Decisions

- Lookup Tables: `taxi_zones` acts entirely as a static lookup table. It is broadcasted during joins.
- Unpartitioned Datasets: `taxi_zones` is not partitioned. Partitioning such a small table would yield numerous tiny files, generating significant metadata overhead that outweighs any benefits.
- Harmful Partitioning: Partitioning is harmful when the choice of partition key is divided into too many pieces relative to the dataset size. This chokes the Spark driver during file listing.
- 20x Volume Increase: If volume increased by 20x, daily partitions for taxi trips might become too large (causing long shuffle times). The strategy would shift to sub-partitioning by `pickup_date` + `pickup_hour` to get appropriately sized partitions.

3. Generic Ingestion Framework

Generic vs. Specific Components

- Generic: Schema validation (`validate_schema`), type compatibility checking (COMPATIBLE dictionary), generic data quality filtering (`quality_filter` which handles nulls, value ranges, and timestamp sequencing), deduplication (`promote`), and standard Delta read/write utilities (`read_bronze`, `write_silver`, etc.).
- Specific: The mapping dictionaries for schema expectations (`expected`), source-specific column renaming (`rename_columns`), and localized timezone casting.

Rule Maintenance and Metadata

- Transformation Rules: Handled via shared mapping dictionaries and centralized helper functions. Schema validation rules are decoupled into a YAML configuration file (`config/compatible_types.yaml`), preventing hardcoded type definitions.
- Metadata Management: Schema versions are managed by Delta Lake's transaction log. Ingestion statistics (processed records, rejected records, schema mismatches) are calculated in the `promote` function and logged.

- Handling 20 New Datasets: The generic framework makes adding 20 new datasets easier. Adding 20 datasets requires only registering their schema dictionaries and invoking `validate_schema` and `promote`. The framework allows for columns to be read based on parameters (`not_null`, `numeric_range`, `timestamp_range`) without requiring custom filters per dataset.

4. Common Data Model

To ensure seamless integration in the Gold layer, the Silver layer enforces a Common Data Model:

Standard Timestamp Format

All event times are converted to `America/New_York` time.

- Weather: Source `ISD DATE (UTC)` is converted via `from_utc_timestamp`.
- Air Quality: Source GMT date/time is combined and converted via `from_utc_timestamp`.
- Taxi Trips: Source TLC timestamps are already naive local time; retained as-is but filtered for valid 2024 bounds.
- Standard analytical date and hour columns (`pickup_date`, `pickup_hour`, `observation_date`, `observation_hour`) are extracted as integers/dates to facilitate rapid join conditions without continuous casting.

Naming Conventions

All source-specific prefixes (`tpep_`, `PULocationID`) are mapped to human-readable `snake_case` (e.g., `pickup_location_id`, `dropoff_location_id`).

Missing Values

Data failing critical structural checks (e.g., null primary keys) are routed to `_rejects` table. In the integration phase, left joins preserve base taxi trips, and missing lookup categories are handled using `.fillna()` with the literal value `"UNKNOWN"` (e.g., for missing boroughs).

Data Types:

- Monetary and physical measurements (`fare_amount`, `temperature_c`, `pm25`) are standardized to `double`.
- Identifiers and counts (`location_id`, `passenger_count`) are typed as `integer` or `long`.
- Temporal fields are strictly typed as `timestamp` or `date`.

5. Integration strategy

The Gold layer pipeline produces `integrated_taxi_trips`, appending context to the tables.

Strategy

- Temporal Joins: Weather and Air Quality data are aggregated into hourly windows (`pickup_date`, `pickup_hour`). The taxi trip fact table is joined against these dimensions using the extracted date and hour of the trip's start time.
- Spatial Joins: The Taxi Zones lookup table is split into two logical views (`pickup_zones` and `dropoff_zones`). Because this dataset is small, it is injected using a Broadcast Join
- Handling Missing Observations: A Left Join ensures that no taxi trips are dropped if the weather or air quality observations did not give a measurement. Categorical metadata missing from the zones table defaults to "UNKNOWN". Weather and air quality values are left as nulls to accurately represent missing data rather than skewing averages with zeros.

6. Decisions and Trade-offs

Hourly Temporal Data

Continuous data (weather, air quality) is grouped and averaged by `pickup_date` and `pickup_hour` prior to the join. This simplifies the join condition into a clean match on two discrete keys (`pickup_date`, `pickup_hour`) and facilitates the comparisons. The trade-off is the loss of minute-level precision, since a taxi trip starting at kl 5:01 receives the same average weather context as a trip starting at kl 5:59.

Strict Left Joins for Data

The pipeline prioritizes keeping all taxi trips by appending the data via left joins and accepting null rows from the weather and air quality data if the context is missing. This prevents the loss of valid taxi trips caused by a faulty sensor. The trade-off is that later, reporting tools must be implemented to handle the UNKNOWN aggregations safely, and cannot rely on fully populated rows.

Isolating Rejected Records

The `promote` function writes rows failing data quality checks into parallel `_rejects` Delta tables rather than dropping them silently or halting the pipeline. This guarantees zero data loss and enables continuous data auditing. The technical cost is an increase in overhead and storage consumption during the promotion phase, as the cluster must evaluate the quality rules and execute an additional write operation.